

THE UNLOX ACADEMY

An Outpro.India Initiative

Data Science · Data Analytics

Week 1 — Master Assignment

Python Fundamentals + NumPy Foundations

50 Questions • Interview-Calibrated

Theory · Output Prediction · Coding Practice

How to Attempt This Assignment

This assignment is engineered to look easy on the surface and bite you on the second read. Every question is something an interviewer might ask in a real product-based or service-based company hiring at the entry level for Data Science / Software roles. The topics are exactly what you have learned in Week 1, but the angles are not.

Read every question twice. Where you feel "this is too simple", that's exactly where the trap is.

Rules of engagement

- Attempt every question. Partial attempts get partial marks.
- For theory questions: answer in your OWN words, in 2–4 lines max. No copy-paste from ChatGPT or Google. We will detect it.
- For output-prediction questions: write the predicted output FIRST, then run the code, then explain WHY it produced that output.
- For coding questions: write clean code with proper variable names, add 1–2 line comments where logic is non-obvious.
- DO NOT use external libraries unless the question explicitly says so. Only numpy where mentioned.
- Do NOT use built-in shortcuts when the question asks you to implement the logic yourself (e.g., no `.reverse()` if the question says "reverse without using `reverse()`").

Submission format

- Single `.ipynb` (Jupyter notebook) or `.py` file per section, named: `SectionA_<YourName>.ipynb`
- Submit Jupyter Notebook link via the LMS form before the deadline.

Interview Insight — Why this assignment matters

In a real interview, the questions on this assignment are the WARM-UP round. If you cannot solve these in 5–10 minutes each without referring to Google, you will not clear screening. Treat this assignment as your personal benchmark, not as homework.

Section A - Theory (Interview Focus)

10 questions. Answer in 2–4 lines each. Brevity is a skill, interviewers don't want lectures. They want clarity.

Q1. What is the difference between the ``is`` operator and the ``==`` operator? Give one example where two variables return True for ``==`` but False for ``is``.

Interview Insight - Why this gets asked

This question tells the interviewer in 30 seconds whether the candidate understands memory references vs value equality. 80% of freshers say "both check equality" and fail.

Q2. Classify the following as Mutable or Immutable: list, tuple, set, dict, str, int, float, bool, frozenset. Explain in one line WHY immutability matters in real code.

Q3. Predict the output of ``print(0.1 + 0.2 == 0.3)``. Then explain WHY this happens. (Hint: it is not a Python bug.)

Q4. Both list and tuple can store ordered collections. Give 3 concrete situations where you would CHOOSE a tuple over a list, and explain the reasoning behind each.

Q5. Why is dictionary lookup $O(1)$ on average while searching for a value in a list using ``in`` is $O(n)$? Explain in plain English without using the word "hash" - then explain it again WITH the word "hash".

Q6. What is the difference between ``range()`` and ``numpy.arange()``? Mention at least 3 differences, one of them should involve floats.

Q7. What is the difference between ``list.append(x)``, ``list.extend(x)``, and ``list.insert(i, x)``? For each, show what happens when ``x = [4, 5]`` is added to ``a = [1, 2, 3]``.

Q8. What does this line do, and what are the values of a, b, c after execution?

```
a, *b, c = [10, 20, 30, 40, 50]
```

Now, what happens if you write: ``a, *b, c = [10, 20]``? And ``a, *b, c = [10]``?

Q9. Explain the difference between ``/``, ``//``, ``%``, and ``**`` operators in Python. For each, give the output of the operation when applied to ``(-7, 2)``.

Q10. Why is NumPy significantly faster than Python lists for numerical operations? Give 3 reasons, at least one should mention memory layout.

IN Industry Connect, Real interviews these come from

Q1, Q5, Q10 - asked at Razorpay, Flipkart, Swiggy data team interviews (2024–25).

Q4, Q7 - asked in Infosys, TCS, Wipro mass-recruitment Python rounds.

Q2, Q9 - asked in product-based screening rounds for backend & data roles.

Section B - Output Prediction (The Tricky Ones)

5 questions. For each: (a) write your predicted output BEFORE running, (b) then run it, (c) then explain WHY in 1–2 lines. If you skip step (a), you score zero for that question.

Q11. Predict the output. Why does the second line behave differently from the first?

```
matrix1 = [[0] * 3 for _ in range(3)]
matrix2 = [[0] * 3] * 3

matrix1[0][0] = 99
matrix2[0][0] = 99

print(matrix1)
print(matrix2)
```

Q12. Predict the output:

```
x = [1, 2, 3]
y = x
x = x + [4]
print("x =", x)
print("y =", y)

a = [1, 2, 3]
b = a
a += [4]
print("a =", a)
print("b =", b)
```

Now answer: Why do `x = x + [4]` and `a += [4]` behave differently?

Q13. Predict the output. What is the value of `i` after the loop ends, and why?

```
for i in range(5):
    pass
print(i)

for j in range(0):
    pass
print(j) # think before you write
```

Q14. Predict the output of each print statement. Then state which lines will raise an error (if any).

```
print("10" + "20")
print(10 + 20)
print("10" * 3)
print(10 * "3")
```

```
print("10" + 20)
print(int("10") + 20)
```

Q15. Predict the output. Then explain Python's short-circuit evaluation in one line.

```
print(0 or 5)
print(5 or 0)
print(None or "hello")
print(0 and 5)
print(5 and 10)
print([] or "default")
print([0] or "default") # careful!
```

 Interview Insight - The trap in Q15

`[0]` is a non-empty list, its truth value is True, not False. `bool([0])` returns True. Many candidates assume "contains zero = falsy" and lose points. Empty container = falsy. Non-empty container = truthy, regardless of contents.

Section C - Coding Practice

35 questions. Difficulty grows progressively. Solve in order. Constraint: use ONLY what is taught in Week 1 - no list comprehension shortcuts unless asked, no map/filter/lambda unless mentioned, no external libraries beyond NumPy where specified.

C1 – Warm-up (Q16-Q20)

Q16. Swap two numbers **WITHOUT** using a third variable. Implement 3 different methods: (a) using arithmetic, (b) using XOR (^), (c) using tuple unpacking. Compare which is most Pythonic.

Q17. Swiggy Order Calculator. Take item price and quantity as input. Compute total = price × quantity. Add 18% GST. Add ₹40 delivery charge if total (before delivery) is below ₹500, else delivery is FREE. Print all components separately and final amount in this format:

```
Subtotal:      ₹450.00
GST (18%):     ₹81.00
Delivery:      ₹40.00
-----
Total:         ₹571.00
```

Q18. Temperature converter. Ask user for a value and unit (C or F). Convert and print result rounded to 2 decimal places. Handle invalid unit input gracefully (print an error message, do NOT crash).

Q19. Take a 4-digit number as input. Print the sum of its digits – **WITHOUT** converting it to a string. (Use only integer arithmetic: //, %.)

Q20. Smart input parser. Take a single input from the user. Detect what type it is – try int first, then float, else treat as string. Print the value AND its detected type. Example: input "42" → int 42, input "3.14" → float 3.14, input "hello" → str hello.

IN Industry Connect – Why Q19 is asked

Q19 (sum of digits without string conversion) is a Wipro, Cognizant, Capgemini favourite. It checks whether the candidate truly understands `//` and `%`, or whether they always reach for `str()` as a crutch. Same trick is used to: reverse a number, count digits, check palindrome, all without strings.

C2 – Conditionals & Logic (Q21-Q25)

Q21. Indian Income Tax Calculator (New Regime, FY 2024–25). Take annual income as input. Compute tax using these slabs:

```
Up to ₹3,00,000      : Nil
₹3,00,001 – ₹7,00,000 : 5%
₹7,00,001 – ₹10,00,000 : 10%
₹10,00,001 – ₹12,00,000 : 15%
₹12,00,001 – ₹15,00,000 : 20%
Above ₹15,00,000     : 30%
```

Tax must be computed slab-wise, NOT flat. Example: income ₹8,00,000 → 0 on first 3L + 5% on next 4L + 10% on last 1L = ₹20,000 + ₹10,000 = ₹30,000. Print breakdown clearly.

Q22. Validate Indian PAN card format WITHOUT using regex. Format: AAAAA9999A, first 5 are uppercase letters, next 4 are digits, last 1 is an uppercase letter. Total length = 10. Take input and print "Valid PAN" or "Invalid PAN — <reason>" where reason explains exactly what's wrong.

Q23. Bengaluru BESCOM Electricity Bill (LT-2 Residential). Take units consumed as input. Charges:

```
0 – 50 units      : ₹4.10 per unit
51 – 100 units    : ₹5.55 per unit
101 – 200 units   : ₹7.10 per unit
Above 200 units   : ₹8.15 per unit
```

Add fixed charge: ₹110. Add 6% government tax on (energy charges + fixed charge). Print itemized bill.

Q24. Cricket Strike Rate Classifier. Take runs scored and balls faced. Compute strike rate = $(\text{runs/balls}) \times 100$. Classify:

```
SR > 150      → Explosive (T20 finisher)
100 < SR ≤ 150 → Aggressive (Top order T20)
75 ≤ SR ≤ 100 → Steady (ODI anchor)
SR < 75       → Defensive (Test match)
```

Edge case: handle balls = 0 (do not crash, print message instead).

Q25. BMI Calculator (Indian standards). Take weight (kg) and height (cm). Compute BMI = $\text{weight} / (\text{height_in_meters})^2$. Classify using Indian (not WHO) BMI categories:

```
BMI < 18.5      → Underweight
18.5 – 22.9     → Normal (Indian range – narrower than WHO)
23.0 – 24.9     → Overweight
25.0 – 29.9     → Obese Class I
BMI ≥ 30        → Obese Class II
```


C3 – Nested Loop Patterns (Q26-Q30)

Take *N (size)* as input where applicable. Use only ``print()``, ``for` / `while``, and ``range()``. No string multiplication shortcuts where the pattern can be drawn with loops.

Q26. Print this pattern for N = 5:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Q27. Print this number diamond for N = 5 (the digits go up to N in the middle row):

```
    1
  1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
  1 2 3 4 3 2 1
    1 2 3 2 1
      1 2 1
        1
```

Q28. Print a hollow square of size N (only the border is *, inside is blank). For N = 5:

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Q29. Print the first 5 rows of Pascal's Triangle:

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

You may use any approach (formula-based or direct calculation), but no `math.comb()` or any imported function.

Q30. Print this character pattern for N = 5 – letters go up to the N-th alphabet:

```
A
A B
A B C
A B C D
A B C D E
```

Hint: ``chr()`` and ``ord()``. Do NOT hardcode a string of letters.

C4 – Loop Logic (Q31-Q36)

Q31. Find all Armstrong numbers between 1 and 9999. (An n-digit number is Armstrong if the sum of its digits each raised to the power n equals the number itself. Example: $153 = 1^3 + 5^3 + 3^3$.) Print them in a clean comma-separated list with a count at the end.

Q32. Take N as input. Print the first N terms of the Fibonacci series. Also compute and print the SUM of those N terms. Do NOT use recursion. Edge case: N = 0 should print nothing, sum = 0. N = 1 should print just 0.

Q33. Prime check, optimized. Take a number N. Check if it is prime by iterating only up to $\sqrt{N}$. Then, print the SUM of all primes between 2 and N (inclusive). Do not use any library.

Q34. Take a number as input. Reverse it WITHOUT converting to string. Then check if the original and reversed are equal (palindrome). Handle negative numbers, they should be considered NOT palindrome (because of the minus sign). Examples: 121 → palindrome, 123 → not palindrome, -121 → not palindrome.

Q35. Take two positive integers. Compute their HCF (Highest Common Factor) using Euclidean algorithm with a `while` loop. Then compute LCM using the relation: $HCF \times LCM = a \times b$. Do not use math.gcd.

Q36. String scanner, single pass. Take a sentence (e.g., a Swiggy review like "Order came on time, food was hot, packaging 10/10!"). In ONE pass through the string, count: total characters, total words, vowels (a, e, i, o, u - case-insensitive), consonants, digits, spaces, and special characters. Print all counts.

Interview Insight – Why "single pass" matters in Q36

Real datasets have millions of characters. If your code iterates through the string 5 times (once per count), it is 5× slower. Interviewers ask "can you do this in one pass?" specifically to test whether you think about efficiency or just write whatever works first.

C5 – Lists, Tuples, Sets, Dicts (Q37-Q42)

Q37. Take 10 student marks (out of 100) as input, space-separated on one line, parse with `split()` and `map()`. Without using `sort()` or `sorted()`, find: (a) maximum, (b) minimum, (c) average, (d) count of students above average, (e) the SECOND highest mark.

Q38. Word Frequency Analyzer for Swiggy reviews. Take a review sentence. Find the frequency of each word (case-insensitive, "Food" and "food" should be counted together). Print words sorted by frequency in descending order. Also print the MOST frequent word and its count. Use a dict.

Q39. Indian States CRUD. Pre-populate a dict with 5 Indian states and their capitals (e.g., Karnataka → Bengaluru, Maharashtra → Mumbai). Then build a menu-driven program:

1. Add a new state-capital pair
2. Update an existing capital
3. Delete a state
4. Search a state (return capital, or 'Not found')
5. Display all states sorted alphabetically
6. Exit

Use a while loop. Loop until user chooses option 6.

Q40. Set Operations. Students in Section A know languages = {Python, Java, C++, SQL}. Students in Section B know = {Java, JavaScript, Python, Go, SQL}. Using ONLY set operations, find:

- (a) Languages known by BOTH sections
- (b) Languages known ONLY by Section A
- (c) Languages known ONLY by Section B
- (d) Total unique languages across both sections
- (e) Languages exclusive to one section but not both (symmetric difference)

Q41. TCS stock prices for a week stored as a tuple: `prices = (3845, 3902, 3878, 3955, 4012, 3998, 4055)` (Mon to Sun in ₹). Find: (a) maximum price and which day, (b) minimum price and which day, (c) average, (d) biggest single-day gain (and between which days), (e) total volatility = sum of absolute day-to-day price changes.

Q42. Given a list of 20 integers (you generate or hardcode them, some duplicates), produce: (a) the set of unique numbers, (b) the list of numbers appearing more than once, (c) the list of numbers appearing EXACTLY once. Do this WITHOUT importing Counter or any other library.

C6 – Strings (Q43-Q45)

Q43. Smart Palindrome Checker. Take a sentence as input. Check if it is a palindrome **IGNORING**: case, spaces, and all special characters (punctuation). Examples that should return TRUE: "A man, a plan, a canal: Panama", "Race car", "No 'x' in Nixon". Hint: build a cleaned version of the string first.

Q44. Manual Title Case. Take a sentence (e.g., "hello world from bengaluru"). Capitalize the first letter of EACH word, but do NOT use .title() or .capitalize(). Use only .upper() / .lower() and string indexing. Output: "Hello World From Bengaluru".

Q45. Caesar Cipher Encryptor. Take a message and a shift value N as input. Encrypt by shifting each letter forward by N positions in the alphabet (wrapping around, Z + 1 = A). Preserve case. Keep digits, spaces, and special characters **UNCHANGED**. Example: "Hello, India!" with shift 3 → "Khoor, Lqgld!"

IN Industry Connect – Where strings get asked

String manipulation questions are 30% of all coding round questions in Indian service-company recruitment (TCS NQT, Wipro NLTH, Infosys InfyTQ). Mastering Q43 alone covers ~5 different palindrome variants you'll see.

C7 – Functions (Q46-Q47)

Q46. Write a function `calculate_gst(amount, gst_rate=18, gst_inclusive=False)`:

- If `gst_inclusive=False`: amount is base, add GST on top
- If `gst_inclusive=True`: amount already includes GST, extract it
- Return a tuple: (base_amount, gst_amount, total_amount)
- All values rounded to 2 decimals

Test cases:

```
calculate_gst(1000)                → (1000.00, 180.00, 1180.00)
calculate_gst(1180, gst_inclusive=True) → (1000.00, 180.00, 1180.00)
calculate_gst(500, gst_rate=5)     → (500.00, 25.00, 525.00)
```

Q47. Write a RECURSIVE function `power(base, exp)` that computes base raised to exp, WITHOUT using the `**` operator and WITHOUT using `pow()`. Handle `exp = 0` (return 1) and negative exp (return `1/power(base, -exp)`). Test with: `power(2, 10)`, `power(5, 0)`, `power(3, -2)`.

C8 – NumPy Foundations (Q48-Q50)

Q48. Identity Matrix Manipulations. Using NumPy: (a) create a 5×5 identity matrix using `np.eye()`, (b) multiply every element by 10, (c) transpose it (yes, it looks the same, explain in a comment WHY transpose of an identity is itself), (d) compute the sum of diagonal elements, (e) compute the sum of all elements. Print each step.

Q49. Zomato Weekly Sales Analytics. Create a NumPy array of 7-day Zomato Bengaluru sales (in ₹):

```
sales = np.array([12000, 18500, 14200, 22100, 19800, 35200, 28400])
               # Mon   Tue    Wed    Thu    Fri    Sat    Sun
```

Without using loops, compute and print:

- (a) Total weekly sales
- (b) Daily average
- (c) The day (Mon/Tue/...) with maximum sales
- (d) Number of days with sales above the weekly average
- (e) Percentage growth from Monday to Sunday
- (f) An array of day-on-day % changes (length 6)

Q50. The Border Pattern Challenge – NO explicit loops. Create a 5×5 NumPy array where the BORDER is 1 and the INTERIOR is 0. Output should look like:

```
[[1 1 1 1 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 0 0 0 1]
 [1 1 1 1 1]]
```

Constraint: You may use `np.ones()`, `np.zeros()`, and slicing, but NO `for`, NO `while`, NO list comprehension. Only NumPy operations.

Interview Insight – Why Q50 is interview gold

This single problem tests: array creation, indexing, slicing, and the vectorized mindset that separates a NumPy user from a Python-with-numpy user. If you reach for a loop here, you have not internalized NumPy yet, go re-read the lecture notes.

Final Notes

If you genuinely solved all 50 questions, you are interview-ready for Week 1 topics. Now flip the table: pick any 5 questions and EXPLAIN your solution out loud, as if to a panel. That's where most candidates fail, they can code, but they cannot articulate.

Common traps I have seen students fall into this week:

- Confusing `=` with `==` (assignment vs comparison).
- Using `range(n)` and thinking it includes `n`. It does NOT, last value is `n-1`.
- Modifying a list while iterating over it, leads to skipped elements.
- Comparing floats with `==` - always use `abs(a-b) < 1e-9`.
- Forgetting that `input()` always returns a string, even if user types digits.
- Treating sets as ordered, they are NOT.
- Believing `numpy` arrays behave like Python lists, they don't (broadcasting, dtype, views vs copies).

IN Industry Connect, Your placement readiness signal

If you solved 45/50 unaided → strong foundation, focus next on data structures (Week 2).

If you solved 30–45 → revise topics where you struggled, redo those questions on day 2.

If you solved <30 → schedule a 1:1 with mentor. Do NOT move to Week 2 yet.

Girish, Lead Mentor, The Unlox Academy

All the best. Code daily. Sleep when you must.